

Python E-Course

Module 9 — Edge Detection

Detailed Notes

Module Overview

Level: Detection

Duration: ~1 week

Prerequisites: Modules 1-8

What You'll Learn

- Compute gradients with Sobel and Scharr.
- Detect edges with Laplacian.
- Run the Canny pipeline end-to-end (Gaussian → gradient → NMS → hysteresis).
- Tune low / high thresholds.

Lessons

#	Lesson	Duration
1	What is an Edge?	25 min
2	Sobel and Scharr Gradients	30 min
3	Laplacian as Edge Detector	25 min
4	Canny — The Full Pipeline	35 min
5	Tuning Canny Thresholds	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 10: Morphological Operations

Lesson 1: What is an Edge?

Module: 9 — Edge Detection

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Define an edge in terms of intensity gradients.
- Distinguish step edges, ramp edges, and ridge edges.
- Identify orientation and magnitude of a gradient.

Prerequisites

- Modules 1-8

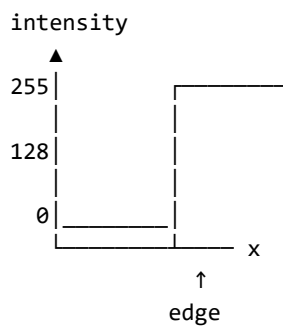
1. Why This Matters

Edges are the most informative pixels in an image — they outline objects. Almost every higher-level task (segmentation, contour detection, object recognition) starts from edges.

2. Concept

2.1 An edge in 1D

Plot intensity along a row:



The derivative (gradient) is large at the edge, small elsewhere.

2.2 Edge types

- Step: a sudden jump from one value to another (synthetic / extreme).
- Ramp: a gradual transition over several pixels (most real edges).
- Ridge: a thin bright line on a dark background — peak surrounded by drops.

2.3 In 2D — gradient = vector

The gradient has two components:

$$\nabla I = (\partial I / \partial x, \partial I / \partial y)$$

- Magnitude: $|\nabla I| = \sqrt{(\partial I / \partial x)^2 + (\partial I / \partial y)^2}$ — strength of the edge.
- Orientation: $\theta = \text{atan2}(\partial I / \partial y, \partial I / \partial x)$ — direction perpendicular to the edge.

2.4 The three classical detectors

Detector	What it measures
Sobel / Scharr	First derivative (gradient magnitude)
Laplacian	Second derivative (zero-crossings = edges)
Canny	Multi-step pipeline using Sobel + non-max suppression + hysteresis

2.5 Pre-blur

Derivatives amplify noise. Always apply a small Gaussian blur ($\sigma \approx 1$) before any edge detector. Canny does this internally.

3. Code Examples

Example 1: 1D gradient of a row

```
import numpy as np
import matplotlib.pyplot as plt

row = np.concatenate([np.full(20, 30), np.full(20, 200), np.full(20,
30)]).astype(np.float32)
grad = np.diff(row)

fig, ax = plt.subplots(2, 1, figsize=(8, 4))
ax[0].plot(row); ax[0].set_title("intensity"); ax[0].set_ylim(0, 255)
ax[1].plot(grad); ax[1].set_title("gradient"); ax[1].axhline(0, color="gray",
lw=0.5)
plt.tight_layout(); plt.show()
print("max |gradient|:", np.abs(grad).max())
```

Expected Output: Top plot: flat-low, jump up, flat-high, jump down, flat-low. Bottom plot: zero everywhere except a positive spike at the rising edge and a negative spike at the falling edge. Console: max |gradient|: 170.

Example 2: Gradient magnitude with NumPy

```
import numpy as np, cv2
from skimage.data import camera

img = camera().astype(np.float32)
gx = np.gradient(img, axis=1)
gy = np.gradient(img, axis=0)
mag = np.sqrt(gx**2 + gy**2)
mag_vis = np.clip(mag, 0, 255).astype(np.uint8)
cv2.imwrite("cam_gradient_mag.png", mag_vis)
print("mag range:", mag.min(), mag.max())
```

Expected Output: cam_gradient_mag.png highlights edges — building outlines, the cameraman silhouette. Console shows max around 200-300.

Example 3: Orientation visualisation

```
import numpy as np, cv2
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (0, 0), 1.0).astype(np.float32)
gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
ang = np.arctan2(gy, gx) * 180 / np.pi    # degrees, [-180, 180]
print("orientation range:", ang.min(), ang.max())
```

Expected Output:

```
orientation range: -180.0 180.0
```

4. Parameter Sensitivity

(Per-detector — see Lessons 2-5.)

5. Common Mistakes

Mistake	What Happens	Fix
Skip pre-blur	Noisy edge map	Gaussian $\sigma=1$ before edge detection
Use uint8 output for gradients	Negative values wrap	Use CV_32F
Treat one gradient axis as the edge	Misses orthogonal edges	Compute magnitude from both axes

6. Practice Tasks

- Plot a 1D step-edge row and its `np.diff`. Identify the spike.
- Compute gradient magnitude of `camera()` via `np.gradient`. Visualise.
- Compute orientation map (degrees); print min/max.

7. Key Takeaways

- Edge = location of rapid intensity change.
- 2D gradient has magnitude (strength) and orientation (perpendicular to edge).
- Always blur slightly first to reduce noise.

8. What's Next

Sobel — the standard discrete first-derivative operator.

Lesson 2: Sobel and Scharr Gradients

Module: 9 — Edge Detection

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply cv2.Sobel and cv2.Scharr for gradients.
- Combine x and y gradients into a magnitude image.
- Know when Scharr beats Sobel.

Prerequisites

- Module 9 Lesson 1

1. Why This Matters

Sobel is the de facto discrete derivative operator. Almost every edge / corner / feature detector uses it under the hood (including Canny). Scharr is a small upgrade with better isotropy at the cost of speed.

2. Concept

2.1 The Sobel kernels (3×3)

Horizontal gradient ($\partial I / \partial x$ — detects vertical edges):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Vertical gradient ($\partial I / \partial y$ — detects horizontal edges):

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The middle row/col weighted by 2 emphasises the centre, which gives better noise rejection than a plain $[-1, 0, 1]$.

2.2 The Scharr kernels (3×3)

$\partial / \partial x:$	$\partial / \partial y:$
$\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$	$\begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}$

Bigger central weights — more rotationally symmetric responses. Slightly more accurate for small kernels.

2.3 OpenCV API

```
gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
```

Or:

```
gx = cv2.Scharr(img, cv2.CV_32F, 1, 0)
gy = cv2.Scharr(img, cv2.CV_32F, 0, 1)
```

dx, dy = 1, 0 means "first derivative in x, zero in y".

2.4 Magnitude and direction

```
mag = cv2.magnitude(gx, gy)          # sqrt(gx2 + gy2)
ang = cv2.phase(gx, gy, angleInDegrees=True)
```

Or just NumPy:

```
mag = np.sqrt(gx**2 + gy**2)
ang = np.arctan2(gy, gx) * 180 / np.pi
```

2.5 Larger kernels

cv2.Sobel(..., ksize=5) or 7 smooths first (acts like Sobel applied to a blurred image). Use larger ksize when you want coarser edge maps.

2.6 Pre-blur recommendation

Even with Sobel's built-in smoothing, a 3×3 Gaussian $\sigma \approx 1$ before gradient gives noticeably cleaner edges on noisy images.

3. Code Examples

Example 1: Sobel X / Y

```
import cv2, numpy as np
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (3, 3), 0)
gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)

cv2.imwrite("sobel_x.png", cv2.convertScaleAbs(gx))
cv2.imwrite("sobel_y.png", cv2.convertScaleAbs(gy))
print("gx range:", gx.min(), gx.max())
print("gy range:", gy.min(), gy.max())
```

Expected Output: sobel_x.png highlights vertical edges; sobel_y.png highlights horizontal edges. Console shows negative and positive ranges.

Example 2: Magnitude

```
import cv2, numpy as np
from skimage.data import coins

img = cv2.GaussianBlur(coins(), (3, 3), 0)
gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
```

```

gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
mag = cv2.magnitude(gx, gy)
mag_vis = np.clip(mag, 0, 255).astype(np.uint8)
cv2.imwrite("coins_mag.png", mag_vis)
print("max mag:", mag.max())

```

Expected Output: coins_mag.png shows clean circular outlines around each coin.

Example 3: Sobel vs Scharr

```

import cv2, numpy as np
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (3, 3), 0)
sb_x = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
sb_y = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
sc_x = cv2.Scharr(img, cv2.CV_32F, 1, 0)
sc_y = cv2.Scharr(img, cv2.CV_32F, 0, 1)

sb_mag = np.clip(np.sqrt(sb_x**2 + sb_y**2), 0, 255).astype(np.uint8)
sc_mag = np.clip(np.sqrt(sc_x**2 + sc_y**2), 0, 255).astype(np.uint8)
cv2.imwrite("sobel_mag.png", sb_mag)
cv2.imwrite("scharr_mag.png", sc_mag)

```

Expected Output: Both show edge maps; Scharr's are slightly stronger and more uniform across orientations.

Example 4: ksize tuning

```

import cv2, numpy as np
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (3, 3), 0)
for k in [3, 5, 7]:
    gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=k)
    gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=k)
    mag = np.clip(np.sqrt(gx**2 + gy**2), 0, 255).astype(np.uint8)
    cv2.imwrite(f"sobel_{k}.png", mag)

```

Expected Output: k=3 has thin precise edges; k=7 has thicker, smoother edges (more spatial integration).

4. Parameter Sensitivity

Param	Effect
ksize	larger = thicker / smoother edges
pre-blur σ	larger = fewer fine-detail responses
Sobel vs Scharr	Scharr more isotropic for ksize=3

5. Common Mistakes

Mistake	What Happens	Fix
Use uint8 ddepth	Negative gradients clipped to 0 → asymmetric response	Use CV_32F

Skip pre-blur	Noisy edge map	Light Gaussian first
Combine via `	gx	+

6. Practice Tasks

- Compute Sobel X, Y, magnitude for coins(). Save all three.
- Compare Sobel vs Scharr magnitudes on camera().
- Try ksize 3, 5, 7; compare edge thickness.

7. Key Takeaways

- Sobel = standard first-derivative operator; pair x + y.
- cv2.magnitude(gx, gy) for edge strength.
- Use CV_32F to keep negative values.
- Scharr is the higher-quality 3×3 alternative.

8. What's Next

Laplacian as a second-derivative edge detector.

Lesson 3: Laplacian as Edge Detector

Module: 9 — Edge Detection

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Use cv2.Laplacian as an isotropic edge detector.
- Find zero-crossings as edge locations.
- Use LoG (Laplacian of Gaussian) for blob and edge detection.

Prerequisites

- Module 9 Lesson 2, Module 8 Lesson 2.

1. Why This Matters

Laplacian gives one isotropic response (vs Sobel's separate x/y gradients). It's the basis for blob detection, scale-space (SIFT pyramid), and Marr-Hildreth edge detection. It's also more noise-sensitive than Sobel — pre-blur is essential.

2. Concept

2.1 Second derivative — zero-crossings = edges

Where the second derivative changes sign, the first derivative has an extremum — that's where the edge is. So edge detection via Laplacian = find zero-crossings of the Laplacian image.

2.2 LoG (Laplacian of Gaussian)

Apply a Gaussian blur first, then Laplacian. Or equivalently, apply the LoG kernel directly (math-equivalent, fewer steps).

Smoothing first matters because the Laplacian amplifies noise; the Gaussian σ controls the scale of edges you detect — small σ for fine edges, large σ for big features.

2.3 OpenCV API

```
blur = cv2.GaussianBlur(img, (0, 0), sigma=1.5)
lap = cv2.Laplacian(blur, cv2.CV_32F, ksize=3)
```

2.4 Visualisation choices

- cv2.convertScaleAbs(lap) — show |laplacian| (loses sign).
- Add bias 128, clip, cast — preserves sign for visualization.
- Threshold |lap| > T for a binary edge map.

2.5 Zero-crossings

```
sign = np.sign(lap)
zero_cross = (np.diff(sign, axis=0) != 0) | (np.diff(sign, axis=1) != 0)
```

A boolean image where sign-changes occur. Combined with $|\text{lap}| > T$ gives precise edge locations.

2.6 When to use

- Want isotropic, scale-aware edges → LoG.
- Want fast, classic edges → Canny (next lesson).
- Want gradient direction → Sobel.

3. Code Examples

Example 1: Basic Laplacian

```
import cv2, numpy as np
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (0, 0), 1.5)
lap = cv2.Laplacian(img, cv2.CV_32F, ksize=3)
lap_vis = cv2.convertScaleAbs(lap)
cv2.imwrite("cam_lap.png", lap_vis)
print("lap range:", lap.min(), lap.max())
```

Expected Output: Edge map with bright outlines on a dark background. Range typically -150 to 150.

Example 2: Threshold |Laplacian|

```
import cv2, numpy as np
from skimage.data import coins

img = cv2.GaussianBlur(coins(), (0, 0), 1.5)
lap = cv2.Laplacian(img, cv2.CV_32F, ksize=3)
mask = (np.abs(lap) > 20).astype(np.uint8) * 255
cv2.imwrite("coins_lap_mask.png", mask)
print("edge pixels:", (mask > 0).sum())
```

Expected Output: Binary edge map of the coins (outlines of each coin).

Example 3: LoG with different scales

```
import cv2, numpy as np
from skimage.data import camera

for sigma in [0.5, 1.5, 4.0]:
    blur = cv2.GaussianBlur(camera(), (0, 0), sigma)
    lap = cv2.Laplacian(blur, cv2.CV_32F, ksize=3)
    cv2.imwrite(f"cam_log_s{sigma}.png", cv2.convertScaleAbs(lap))
```

Expected Output: $\sigma=0.5$ catches fine detail (texture, grain). $\sigma=4.0$ catches only coarse structures (building outlines, figure).

Example 4: Zero-crossings

```
import cv2, numpy as np
from skimage.data import coins
```

```

img = cv2.GaussianBlur(coins(), (0, 0), 1.5).astype(np.float32)
lap = cv2.Laplacian(img, cv2.CV_32F, ksize=3)

sign = np.sign(lap)
zc = np.zeros_like(lap, dtype=np.uint8)
zc[:-1] |= (sign[:-1] * sign[1:] < 0).astype(np.uint8)
zc[:, :-1] |= (sign[:, :-1] * sign[:, 1:] < 0).astype(np.uint8)
zc = (zc & (np.abs(lap) > 10).astype(np.uint8)) * 255
cv2.imwrite("coins_zerocross.png", zc)

```

Expected Output: Thin precise edges following the coin boundaries.

4. Parameter Sensitivity

Param	Effect
pre-blur σ	small = fine edges; large = blob-scale edges
ksize	small = local response; large = smoother
threshold	how strong an edge needs to be

5. Common Mistakes

Mistake	What Happens	Fix
No pre-blur	Lots of noise spikes	Always Gaussian first
Use uint8 ddepth	Negative parts lost	CV_32F
Use Laplacian for orientation	It's scalar; no direction	Use Sobel

6. Practice Tasks

- Apply LoG ($\sigma=1.5$) on camera(); visualise.
- Threshold $|\text{Laplacian}| > 20$ on coins(). Save mask.
- Compute zero-crossings; compare to threshold-only edges.

7. Key Takeaways

- Laplacian = second derivative, isotropic.
- Pre-blur with Gaussian — LoG is the standard pairing.
- Zero-crossings or $|\text{lap}| > T$ for binary edges.
- For most practical work, Canny (next lesson) is the default; LoG is a useful alternative for scale-space.

8. What's Next

Canny — the multi-step state-of-the-art edge detector.

Lesson 4: Canny — The Full Pipeline

Module: 9 — Edge Detection

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Run `cv2.Canny` with sensible thresholds.
- Explain the 4 stages: Gaussian → gradient → non-max suppression → hysteresis.
- Visualise each stage independently.

Prerequisites

- Module 9 Lessons 1-3

1. Why This Matters

Canny is the most-used edge detector in the world. It produces thin, well-localised, connected edges with controllable strictness — the gold standard for general edge detection.

2. Concept

2.1 The four stages

- Smooth with Gaussian to reduce noise.
- Gradient via Sobel: magnitude + orientation.
- Non-maximum suppression (NMS): thin the gradient ridges to 1-pixel-wide edges by keeping only local maxima along the gradient direction.
- Hysteresis thresholding:
 - Pixels above high threshold → definite edges.
 - Pixels below low threshold → discarded.
 - Pixels between → kept only if connected (via 8-neighbours) to a definite edge.

The hysteresis step is what makes Canny robust: weak edges that are part of a continuous strong contour survive; isolated weak edges (noise) are dropped.

2.2 OpenCV API

```
edges = cv2.Canny(img, threshold1, threshold2, apertureSize=3, L2gradient=False)
```

- `threshold1, threshold2` = low, high (OpenCV accepts either order; conventionally `low < high`).
- `apertureSize` = Sobel kernel size (3, 5, or 7).
- `L2gradient=True` uses the true $\sqrt{g_x^2 + g_y^2}$ instead of the cheaper $|g_x| + |g_y|$. Slightly more accurate.

2.3 Output

Binary `uint8`: 0 (non-edge) or 255 (edge). Already thinned to one pixel wide.

2.4 Picking thresholds

A common heuristic:

```
v = np.median(img)
low = int(max(0, 0.66 * v))
high = int(min(255, 1.33 * v))
edges = cv2.Canny(img, low, high)
```

Sets the band around the image's median brightness. Works decently for many images; for tuned applications, expose them as parameters.

2.5 Implementation note

OpenCV already does Sobel + NMS + hysteresis. You don't need to chain manual stages — but visualising them helps understanding.

3. Code Examples

Example 1: Basic Canny

```
import cv2
from skimage.data import camera

img = camera()
edges = cv2.Canny(img, 100, 200)
cv2.imwrite("cam_canny.png", edges)
print("edge pixel %:", (edges > 0).mean() * 100)
```

Expected Output: cam_canny.png shows thin clean edges — the cameraman's silhouette, building outlines. Console prints a percentage usually 3-8%.

Example 2: Visualise the four stages manually

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (5, 5), 1.4)
gx = cv2.Sobel(blur, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(blur, cv2.CV_32F, 0, 1, ksize=3)
mag = cv2.magnitude(gx, gy)
mag_vis = np.clip(mag, 0, 255).astype(np.uint8)
edges = cv2.Canny(img, 100, 200)

stages = np.hstack([img, blur, mag_vis, edges])
cv2.imwrite("canny_stages.png", stages)
```

Expected Output: 4-up image: original | blurred | gradient magnitude | final Canny edges.

Example 3: Auto-threshold via median

```
import cv2, numpy as np
from skimage.data import coins

img = coins()
```

```

v = np.median(img)
low, high = int(max(0, 0.66 * v)), int(min(255, 1.33 * v))
edges = cv2.Canny(img, low, high)
print(f"auto: low={low} high={high}, edge %={ (edges > 0).mean()*100:.2f}")
cv2.imwrite("coins_canny_auto.png", edges)

```

Expected Output: coins_canny_auto.png with clean coin outlines. Console prints chosen thresholds.

Example 4: L2 vs L1 gradient

```

import cv2
from skimage.data import camera

img = camera()
e1 = cv2.Canny(img, 100, 200, L2gradient=False)
e2 = cv2.Canny(img, 100, 200, L2gradient=True)
print("L1 edges:", (e1>0).sum(), " L2 edges:", (e2>0).sum())
cv2.imwrite("canny_L1.png", e1)
cv2.imwrite("canny_L2.png", e2)

```

Expected Output: Slightly different counts; L2 typically picks up a few extra subtle edges.

4. Parameter Sensitivity

Param	Lower	Higher
threshold1 (low)	more weak edges kept	fewer weak edges
threshold2 (high)	more edges accepted as definite	only strong edges accepted
apertureSize	fine edges	thicker / smoother edges
L2gradient	faster, less accurate	slower, more accurate

5. Common Mistakes

Mistake	What Happens	Fix
low > high	OpenCV swaps; works but unclear	Set low < high
Thresholds too low	Noisy "edge spaghetti"	Increase both
Thresholds too high	Missing edges	Decrease both
Apply Canny on color image	OpenCV converts to gray automatically	Be explicit: cv2.cvtColor(..., COLOR_BGR2GRAY) first

6. Practice Tasks

- Apply cv2.Canny(camera, 100, 200). Save.
- Build the auto-threshold helper using median.
- Visualise the 4 stages of Canny side-by-side.

7. Key Takeaways

- 4 stages: Gaussian → Sobel → NMS → hysteresis.
- cv2.Canny(img, low, high) runs all of them.

- Auto-thresholds from the image median is a good default.
- Output is binary (0/255), thinned.

8. What's Next

Tuning Canny thresholds in practice — what to do when defaults aren't enough.

Lesson 5: Tuning Canny Thresholds

Module: 9 — Edge Detection

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Build a trackbar tuner for Canny.
- Use Otsu's method to pick thresholds automatically.
- Recognize the symptom-cause table for bad edge maps.

Prerequisites

- Module 9 Lesson 4

1. Why This Matters

Most real Canny work is tuning — image content, lighting, noise all change what's "right." A 20-line trackbar tool saves hours.

2. Concept

2.1 Symptoms and fixes

You see	Likely cause	Fix
Lots of fragmented noise edges	Thresholds too low	Raise both, especially high
Missing edges that should be there	Thresholds too high	Lower both, especially low
Thick / fuzzy edges	apertureSize too large	Use 3
Object outlines broken into segments	Image too noisy → NMS unstable	Pre-blur more (Gaussian $\sigma \sim 2$)
Texture interfering with real edges	Pre-blur too small	Increase pre-blur σ

2.2 Otsu-derived thresholds

Run Otsu's binary threshold on the smoothed image to get a "natural" gray-level split, then use that as high:

```
import cv2
blur = cv2.GaussianBlur(img, (5, 5), 0)
otsu, _ = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
high = otsu
low = 0.5 * high
edges = cv2.Canny(blur, low, high)
```

(Otsu is covered fully in M11.)

2.3 Tracker tuner pattern

```
cv2.createTrackbar("low", "tune", 50, 255, lambda v: None)
cv2.createTrackbar("high", "tune", 150, 255, lambda v: None)
while True:
    low = cv2.getTrackbarPos("low", "tune")
    high = cv2.getTrackbarPos("high", "tune")
    edges = cv2.Canny(img, low, high)
    cv2.imshow("tune", edges)
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break
```

2.4 Multi-scale Canny

For mixed-scale content (small text + big shapes), run Canny at two pre-blur scales and OR the results:

```
e_fine = cv2.Canny(cv2.GaussianBlur(img, (3, 3), 1), low, high)
e_coarse = cv2.Canny(cv2.GaussianBlur(img, (7, 7), 3), low, high)
combined = cv2.bitwise_or(e_fine, e_coarse)
```

2.5 Recommended starting recipe

```
import cv2, numpy as np
def auto_canny(img, sigma=0.33):
    v = float(np.median(img))
    low = int(max(0, (1.0 - sigma) * v))
    high = int(min(255, (1.0 + sigma) * v))
    return cv2.Canny(img, low, high)
```

3. Code Examples

Example 1: Threshold tracker

```
import cv2, sys
from skimage.data import camera

img = camera()
cv2.namedWindow("tune")
cv2.createTrackbar("low", "tune", 50, 255, lambda v: None)
cv2.createTrackbar("high", "tune", 150, 255, lambda v: None)

while True:
    lo = cv2.getTrackbarPos("low", "tune")
    hi = cv2.getTrackbarPos("high", "tune")
    edges = cv2.Canny(img, lo, hi)
    cv2.imshow("tune", edges)
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break
cv2.destroyAllWindows()
```

Expected Interaction: Drag thresholds in real time; observe how more / fewer edges appear.

Example 2: Otsu-derived

```
import cv2
from skimage.data import camera
```

```

img = camera()
blur = cv2.GaussianBlur(img, (5, 5), 0)
otsu, _ = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
edges = cv2.Canny(blur, otsu * 0.5, otsu)
cv2.imwrite("cam_canny_otsu.png", edges)
print("otsu threshold:", otsu)

```

Expected Output: cam_canny_otsu.png with reasonable edges. Otsu's threshold typically lands in the 90-130 range for typical images.

Example 3: auto_canny function

```

import cv2, numpy as np
from skimage.data import coffee

def auto_canny(img, sigma=0.33):
    v = float(np.median(img))
    low = int(max(0, (1.0 - sigma) * v))
    high = int(min(255, (1.0 + sigma) * v))
    return cv2.Canny(img, low, high)

gray = cv2.cvtColor(cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR), cv2.COLOR_BGR2GRAY)
cv2.imwrite("coffee_canny_auto.png", auto_canny(gray))

```

Expected Output: Coffee bean outlines on black.

Example 4: Multi-scale

```

import cv2
from skimage.data import camera

img = camera()
e_fine = cv2.Canny(cv2.GaussianBlur(img, (3, 3), 1), 60, 150)
e_coarse = cv2.Canny(cv2.GaussianBlur(img, (7, 7), 3), 60, 150)
combined = cv2.bitwise_or(e_fine, e_coarse)
cv2.imwrite("cam_canny_multi.png", combined)

```

Expected Output: Edge map that catches both small text and large object outlines.

4. Parameter Sensitivity

Symptom	Try
Too few edges	Halve low
Too many "spaghetti" edges	Raise both thresholds 50%
Broken thin lines	Reduce low only
Texture noise	Pre-blur with larger σ

5. Common Mistakes

Mistake	What Happens	Fix
Tuning on one image, deploying on others	Brittle	Use auto / Otsu thresholds
Combining Canny with very	Edge overlap mess	Use multi-scale OR carefully

different scales naively		
Setting low $\approx$ high	No hysteresis benefit	Keep low $\approx 0.5 \times$ high

6. Practice Tasks

- Build the trackbar tuner; find good thresholds for astronaut().
- Apply the Otsu-derived recipe on camera().
- Implement auto_canny; compare to manual.

7. Key Takeaways

- Tune via trackbar for one-off images.
- Use Otsu or median-based auto-thresholds in pipelines.
- Symptom $\rightarrow$ fix table guides what to twist.
- Multi-scale Canny for mixed-detail content.

8. What's Next

End of Module 9. Next: Module 10 — Morphological Operations.

Module 9 — Exercises

15 exercises.

9.1 What is an edge?

- (E) Plot 1D step-edge intensity and its `np.diff`.
- (M) Compute gradient magnitude of `camera()` with `np.gradient`.
- (H) Compute and visualise gradient orientation (degrees) for `coins()`.

9.2 Sobel

- (E) Sobel X / Y on `camera()`; save both.
- (M) Compute magnitude with `cv2.magnitude` on `coins()`.
- (H) Compare Sobel vs Scharr magnitudes visually.

9.3 Laplacian

- (E) LoG ($\sigma=1.5$) on `camera()`; save.
- (M) Threshold `|Laplacian|>20` on `coins()`.
- (H) Compute zero-crossings; compare to threshold edges.

9.4 Canny

- (E) `cv2.Canny(camera, 100, 200)`. Save.
- (M) Auto-threshold via median; apply on `coffee()`.
- (H) Visualise the 4 Canny stages side-by-side.

9.5 Tuning

- (E) Build the trackbar tuner for `astronaut()`.
- (M) Otsu-derived thresholds on `camera()`.
- (H) Multi-scale OR of two pre-blur Cannys.

Module 9 — Quiz

10 MCQs.

Q1

Edges are characterised by: A. low intensity B. high gradient magnitude C. uniform color D. random noise

Answer: B (L1)

Q2

Sobel kernel detects: A. brightness B. first derivative C. second derivative D. color

Answer: B (L2)

Q3

For an isotropic edge response (one operator, not two), use: A. Sobel B. Scharr C. Laplacian D. Prewitt

Answer: C (L3)

Q4

Why pre-blur before edge detection? A. reduces noise amplification B. compresses C. changes color D. shrinks image

Answer: A (L1, L3)

Q5

Canny stages, in order: A. NMS → Gaussian → gradient → hysteresis B. Gaussian → gradient → NMS → hysteresis C. Gradient → Gaussian → hysteresis → NMS D. Hysteresis → NMS → Gradient → Gaussian

Answer: B (L4)

Q6

Canny's hysteresis stage: A. removes weak edges entirely B. keeps weak edges only if connected to strong edges C. amplifies all edges D. blurs edges

Answer: B (L4)

Q7

For cv2.Canny, the typical low:high ratio is: A. 1:1 B. 1:2 to 1:3 C. 1:10 D. 10:1

Answer: B (L4, L5)

Q8

LoG combines: A. Sobel + Laplacian B. Gaussian + Laplacian C. Sobel + Gaussian D. Canny + Sobel

Answer: B (L3)

Q9

Output of cv2.Canny is: A. grayscale B. RGB C. binary uint8 (0/255), thinned D. float

Answer: C (L4)

Q10

Symptom: too many noisy "spaghetti" edges. Fix: A. lower thresholds B. raise both thresholds C. apply more sharpening D. remove blur

Answer: B (L5)

Module 9 — Assignment: Multi-Method Edge Comparator

Estimated time: 1.5 hours

Combines: Module 9 + Modules 6-7

Brief

Build a CLI that runs Sobel, Laplacian, and Canny on a given image and saves a side-by-side comparison.

Requirements

- `edges.py INPUT [--output OUT] [--canny-low 100] [--canny-high 200]`
- Pre-blur with Gaussian $\sigma=1$.
- Compute Sobel magnitude (visualise), Laplacian (`|abs|`), Canny.
- Concatenate horizontally: original | sobel | laplacian | canny.
- Save as a single image. Print edge-pixel percentages for Canny.

Constraints

- Modules 1-9 only.
- Use argparse.

Expected Output

```
$ python edges.py coins.png
canny edges: 4.8%
saved -> coins_edges.png
```

`coins_edges.png` is a horizontal strip of 4 panels.

Grading Rubric

Criterion	Weight
All 3 methods correct	35%
Visualisation correct	25%
Argparse + flags	15%
Stats reporting	10%
Style	15%

Submission

Save as `assignment_module9.py`.

Module 9 — Mini Project: Auto-Canny with Threshold Sliders

Overview

A Canny tuner with low/high sliders, plus a button-key for auto-threshold mode (median or Otsu).

Concepts Used

- Canny (M9 L4)
- Auto-threshold (M9 L5)
- Trackbars (M2 L3)

Features

- Sliders for low, high, pre-blur $\sigma \times 10$.
- a cycles between manual / median-auto / Otsu-auto modes.
- s saves; q quits.
- Window shows original | edges side-by-side, with mode label.

Starter Code

canny_tuner.py:

```
import sys, cv2, numpy as np

MODES = ["manual", "median-auto", "otsu-auto"]

def auto_thresholds(img, mode):
    if mode == "median-auto":
        v = float(np.median(img))
        return int(max(0, 0.66*v)), int(min(255, 1.33*v))
    if mode == "otsu-auto":
        otsu, _ = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        return int(0.5*otsu), int(otsu)
    return None

def run(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 700: img = cv2.resize(img, None, fx=0.5, fy=0.5)

    cv2.namedWindow("canny")
    cv2.createTrackbar("low", "canny", 50, 255, lambda v: None)
    cv2.createTrackbar("high", "canny", 150, 255, lambda v: None)
    cv2.createTrackbar("blur sigma x10", "canny", 10, 50, lambda v: None)
    mode_idx = 0

    while True:
        sig = max(0.1, cv2.getTrackbarPos("blur sigma x10", "canny") / 10)
        blurred = cv2.GaussianBlur(img, (0, 0), sig)
        mode = MODES[mode_idx]
```

```

if mode == "manual":
    lo = cv2.getTrackbarPos("low", "canny")
    hi = cv2.getTrackbarPos("high", "canny")
else:
    lo, hi = auto_thresholds(blurred, mode)
edges = cv2.Canny(blurred, lo, hi)
combo = np.hstack([img, edges])
cv2.putText(combo, f"[{mode}] low={lo} high={hi} sig={sig:.1f}",
            (10, 20), cv2.FONT_HERSHEY_SIMPLEX, 0.5, 200, 1)
cv2.imshow("canny", combo)
k = cv2.waitKey(30) & 0xFF
if k == ord('q'): break
if k == ord('a'): mode_idx = (mode_idx + 1) % len(MODES)
if k == ord('s'): cv2.imwrite("edges.png", edges); print("saved edges.png")
cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/camera.png")

```

Expected Interaction

Drag sliders, press a to cycle modes, s to save the current edges.

Extension Challenges

- Add a "histogram of edge angles" panel.
- Add multi-scale toggle (m).
- Overlay edges on the original in red for color visualisation.

Grading Rubric

Criterion	Weight
Manual mode works	25%
Auto modes work	25%
Sigma slider	15%
Mode label	10%
Style	25%